

SKILL WEEK-6

SESSION-11

Task1: To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

```
madhulika@LAPTOP-D2M8NKBH:~$ cd ~/OSSP/skill
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill$ mkdir -p week6/task1
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill$ cd week6/task1
pwd
/home/madhulika/OSSP/skill/week6/task1
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week6/task1$ nano history_buffer.c
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week6/task1$ gcc history_buffer.c -o
T1
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week6/task1$ ls -l
total 24
-rwxr-xr-x 1 madhulika madhulika 16520 Sep 16 17:55 T1
-rw-r--r-- 1 madhulika madhulika 3640 Sep 16 17:55 history_buffer.c
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week6/task1$ ./T1
=== Skill Week 6 - Task 1 ===
Command History Buffer

1. Store Command
2. Retrieve Entry
3. Display History
4. Validate Consistency
5. Exit
Enter choice: 1
Enter command: ls
Command stored successfully.

1. Store Command
2. Retrieve Entry
3. Display History
4. Validate Consistency
5. Exit
Enter choice: i
Enter command: Command cannot be empty.

1. Store Command
2. Retrieve Entry
3. Display History
4. Validate Consistency
5. Exit
Enter choice: 1
Enter command: date
Command stored successfully.
```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_COMMAND 100

typedef struct {
    char commands[MAX_HISTORY][MAX_COMMAND];
    int count;
} HistoryBuffer;

void initialize_history(HistoryBuffer *history)
{
    history->count = 0;

    for (int i = 0; i < MAX_HISTORY; i++) {
        history->commands[i][0] = '\0';
    }
}

int add_command(HistoryBuffer *history, const char *command)
{
    if (history->count >= MAX_HISTORY) {
        printf("History buffer full. Removing oldest command.\n");

        for (int i = 1; i < MAX_HISTORY; i++) {
            strcpy(history->commands[i - 1],
                history->commands[i]);
        }

        strcpy(history->commands[MAX_HISTORY - 1], command);

        return 0;
    }

    strcpy(history->commands[history->count], command);
}

```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

```

    }
    break;

    case 2:
        printf("Enter history entry number: ");
        scanf("%d", &position);
        getchar();

        retrieve_entry(&history, position);
        break;

    case 3:
        display_history(&history);
        break;

    case 4:
        if (validate_consistency(&history)) {
            printf("History buffer is consistent.\n");
        } else {
            printf("History buffer consistency check failed.\n");
        }
        break;

    case 5:
        printf("\nProgram terminated successfully.\n");
        return 0;

    default:
        printf("Invalid choice.\n");
}

return 0;
}

```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

Observation: The program successfully stored commands in a fixed-size history buffer and maintained the stored entries according to the defined capacity. When the buffer reached its maximum capacity, the oldest command was removed to make space for the new command. The program successfully displayed all stored commands, retrieved a particular history entry using its position, rejected invalid entry numbers, and handled empty commands appropriately. The consistency validation confirmed that the number of stored commands and the contents of the history buffer were maintained correctly.

Task2: To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

```
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week6/task1$ cd ~/OSSP/skill
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill$ mkdir -p week6/task2
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill$ cd week6/task2
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week6/task2$ nano pipeline.c
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week6/task2$ gcc pipeline.c -o T2
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week6/task2$ ls -l
total 28
-rwxr-xr-x 1 madhulika madhulika 16936 Sep 16 18:01 T2
-rw-r--r-- 1 madhulika madhulika 4806 Sep 16 18:01 pipeline.c
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week6/task2$ ./T2
=== Skill Week 6 - Task 2 ===
Pipeline Structure and Process Connection

1. Store Command
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Exit
Enter choice: 1
Enter command: echo Hello World
Command stored successfully.

1. Store Command
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Exit
Enter choice: 1
Enter command: wc -w
Command stored successfully.

1. Store Command
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Exit
Enter choice: 2

--- Pipeline Layout ---
Stage 1: echo Hello World
```

```

GNU nano 0.7.1 pipeline.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 20
#define MAX_LENGTH 200

typedef struct {
    char command[MAX_LENGTH];
} PipelineCommand;

typedef struct {
    PipelineCommand commands[MAX_COMMANDS];
    int count;
} Pipeline;

void initialize_pipeline(Pipeline *pipeline)
{
    pipeline->count = 0;
}

int add_command(Pipeline *pipeline, const char *command)
{
    if (pipeline->count >= MAX_COMMANDS) {
        printf("Pipeline capacity reached.\n");
        return 0;
    }

    strcpy(pipeline->commands[pipeline->count].command, command);
    pipeline->count++;

    return 1;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify

```

```

    }
    }
    break;

case 2:
    display_pipeline(&pipeline);
    break;

case 3:
    if (validate_pipeline(&pipeline)) {
        printf("Pipeline layout is valid.\n");
    } else {
        printf("Pipeline layout is invalid.\n");
    }
    break;

case 4:
    if (validate_pipeline(&pipeline)) {
        execute_pipeline(&pipeline);
    } else {
        printf("Cannot execute an invalid pipeline.\n");
    }
    break;

case 5:
    printf("\nProgram terminated successfully.\n");
    return 0;

default:
    printf("Invalid choice.\n");
}

return 0;
}
}

```

```

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify

```

Observation: The program successfully created and maintained a pipeline structure in which commands were stored in their specified execution order. The pipeline layout was displayed correctly with each command represented as a separate stage. The program validated whether the pipeline contained valid commands and rejected an empty or invalid pipeline. Using `fork()`, `pipe()`, `dup2()`, and `execvp()`, the processes were created and connected so that the output of one command was passed as the input to the next command. Both two-stage and three-stage pipeline configurations were tested successfully, confirming the correct connection and execution of the pipeline processes.

SESSION-12

Task1: To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

```
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week6/task1$ nano pipe_redirect.c
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week6/task1$ gcc pipe_redirect.c -o T
1
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week6/task1$ ls -l
total 24
-rwxr-xr-x 1 madhulika madhulika 16344 Sep 17 04:42 T1
-rw-r--r-- 1 madhulika madhulika 3640 Sep 16 17:55 history_buffer.c
-rw-r--r-- 1 madhulika madhulika 1713 Sep 17 04:42 pipe_redirect.c
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week6/task1$ ./T1
=== Skill Week 6 - Task 1 ===
Pipe Creation and Data Flow

Pipe created successfully.
25

Both processes completed successfully.
Pipe descriptors closed.
Data flow test completed.
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week6/task1$ pwd
/home/madhulika/OSSP/skill/week6/task1
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week6/task1$
```

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    printf("=== Skill Week 7 - Task 1 ===\n");
    printf("Pipe Creation and Data Flow\n\n");

    /* Create pipe */
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    printf("Pipe created successfully.\n");

    /* Create first child */
    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid1 == 0)
    {
        /* Child 1 writes into pipe */

        close(pipefd[0]);

```

[^]G Help [^]O Write Out [^]F Where Is [^]K Cut [^]T Execute
[^]X Exit [^]R Read File [^]\ Replace [^]U Paste [^]J Justify

```

GNU nano 8.7.1      pipe_redirect.c *
}

if (pid2 == 0)
{
    /* Child 2 reads from pipe */

    close(pipefd[1]);

    if (dup2(pipefd[0], STDIN_FILENO) == -1)
    {
        perror("dup2");
        exit(1);
    }

    close(pipefd[0]);

    execlp("wc", "wc", "-c",
           (char *)NULL);

    perror("execlp");
    exit(1);
}

/* Parent closes both descriptors */
close(pipefd[0]);
close(pipefd[1]);

/* Synchronize child processes */
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

printf("\nBoth processes completed successfully.\n");
printf("Pipe descriptors closed.\n");
printf("Data flow test completed.\n");

return 0;
}

```

[^]G Help [^]O Write Out [^]F Where Is [^]K Cut [^]T Execute
[^]X Exit [^]R Read File [^]\ Replace [^]U Paste [^]J Justify

Observation: The program successfully created a pipe to establish communication between two child processes. The first child process redirected its standard output to the pipe using `dup2()` and executed the `echo` command, while the second child process redirected its standard input from the pipe and executed the `wc` command. The parent process closed the unused pipe descriptors and used `waitpid()` to synchronize with both child processes. The data produced by the first process was successfully transferred through the pipe and processed by the second process, confirming the correct data flow and process synchronization.

Task2: To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

```
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week6/task1$ cd ~/OSSP/skill
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill$ mkdir -p week6/task2
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill$ cd week6/task2
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week6/task2$ ls -l
total 28
-rwxr-xr-x 1 madhulika madhulika 16936 Sep 16 18:01 T2
-rw-r--r-- 1 madhulika madhulika 4806 Sep 16 18:01 pipeline.c
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week6/task2$ ./T2
=== Skill Week 6 - Task 2 ===
Pipeline Structure and Process Connection

1. Store Command
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Exit
Enter choice: 1
Enter command: echo Hello World
Command stored successfully.

1. Store Command
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Exit
Enter choice: 1
Enter command: wc -w
Command stored successfully.

1. Store Command
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Exit
```

```
1. Store Command
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Exit
Enter choice: 4
2

Pipeline execution completed.

1. Store Command
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Exit
Enter choice: 5

Program terminated successfully.
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week6/task2$
```

Observation: The program successfully created and maintained a pipeline structure by storing multiple commands in the required execution order. The stored commands were displayed as separate pipeline stages, and the pipeline layout was validated before execution. The processes were connected using pipes, allowing the output of one command to be passed as input to the next command. The two-stage pipeline was executed successfully, and the resulting output confirmed that data was transferred correctly between the processes. The program also handled invalid and empty pipeline configurations appropriately.